

## Lab Task 2

### Problem 1:

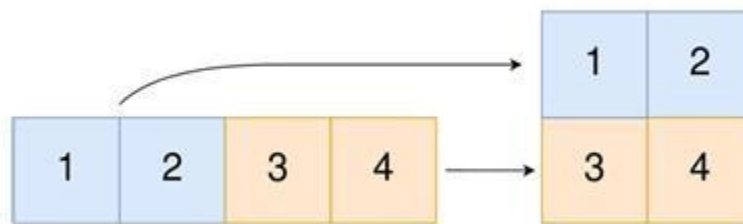
Write a function that finds and returns the largest element in a dynamically allocated 2D array. Use pointers for array traversal.

**Function Prototype:** `int findlargest(int **matrix, int rows, int columns)`

### Problem 2:

You are given a 0-indexed 1-dimensional (1D) integer array `original`, and two integers, `m` and `n`. You are tasked with creating a 2-dimensional (2D) array with `m` rows and `n` columns using all the elements from the `original`. The elements from indices 0 to `n - 1` (inclusive) of `original` should form the first row of the constructed 2D array, the elements from indices `n` to `2 * n - 1` (inclusive) should form the second row of the constructed 2D array, and so on. Return an `m x n` 2D array constructed according to the above procedure, or an empty 2D array if it is impossible.

**Function Prototype:** `Int** construct2DArray(int* original, int m, int n)` **Example 1:**



**Input:** `original = [1,2,3,4]`, `m = 2`, `n = 2`

**Output:** `[[1,2],[3,4]]`

**Explanation:** The constructed 2D array should contain 2 rows and 2 columns. The first group of `n=2` elements in `original`, `[1,2]`, becomes the first row in the constructed 2D array. The second group of `n=2` elements in `original`, `[3,4]`, becomes the second row in the constructed 2D array.

### Problem 3:

Given a m x n matrix grid which is sorted in non-increasing order both row-wise and columnwise, return the number of negative numbers in a grid.

**Function Prototype:** `int countNegatives(int** grid, int rows, int columns)`

#### Example 1

Input grid = `[[4,3,2,-1],[3,2,1,-1],[1,1,-1,-2],[-1,-1,-2,-3]]`

Output 8

**Explanation** There are 8 negatives number in the matrix.

#### Example 2

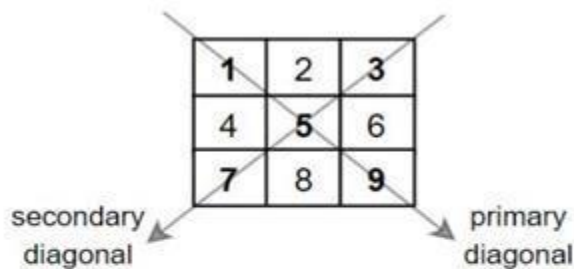
Input grid = `[[3,2],[1,0]]`

Output 0

### Problem 4:

Given a matrix, return the sum of the matrix diagonals. Only include the sum of all the elements on the primary diagonal and all the elements on the secondary diagonal that are not part of the primary diagonal.

**Function Prototype:** `int diagonalSum(int** matrix, int size)` **Example:**



**Input:** matrix = `[[1,2,3],  
[4,5,6],  
[7,8,9]]`

**Output:** 25

**Explanation:** Diagonals sum:  $1 + 5 + 9 + 3 + 7 = 25$

Notice that element `matrix[1][1] = 5` is counted only once.

## Problem 05

You are given a 3×3 grid where 1 shows a live enemy and 0 shows a dead one. Write a C++ program that initializes this grid and first counts how many live enemies are present. Then, repeatedly ask the user to enter row and column indices (0–2). If a live enemy is found at that position, mark it as dead and count it as a hit. Keep taking input until all enemies are destroyed. Finally, display the total number of hits and print the updated grid.

```
1 0 1          1 = Live enemy 0 = Dead enemy
1 1 0
0 1 1
```

```
Enter row and column (0-2): 0 0
```

```
Enter row and column(0-2): 0 2
```

```
Total live enemies: 6
```

```
Hit. Enemy destroyed.
```

```
Hit. Enemy destroyed.
```

```
All enemies are destroyed.
```

```
Total hits required: 6
```

```
Final Grid:
```

```
0 0 0
```

```
0 0 0
```

```
0 0 0
```